课程笔记

CS336 第 17 讲：对齐：RL II

Codex 基于 Stanford CS336 2025 第 17 讲视频、字幕与课程材料整理

2026 年 4 月 9 日

视频作者/频道：Stanford Online
发布日期：2025 年 07 月 08 日
视频时长：1:16:08
视频链接：`https://www.youtube.com/watch?v=JdGFdViaOJk`

目录

1 把语言模型写成一个 RL 问题：state、action、reward 到底是什么 2

1.1 第 17 讲先把符号和对象理顺 2

1.2 本章小结 2

2 naive policy gradient：公式很短，方差很大 2

2.1 从目标函数到 log-derivative trick 2

2.2 为什么 naive 版本会很糟 3

2.3 本章小结 3

3 baseline 与 advantage：为什么“比平均好多少”比“拿了多少分”更重要 3

3.1 baseline 的作用是降方差，而不是改目标 3

3.2 课程那个两状态例子到底想说明什么 3

3.3 advantage 其实就是一个更有语义的 baselined reward 3

3.4 本章小结 4

4 GRPO：把多回答组结构变成廉价 advantage 4

4.1 为什么语言模型天然适合 group-relative baseline 4

4.2 为什么旧策略的 log prob 必须被冻结 5

4.3 KL 正则为什么还要保留 5

4.4 本章小结 5

5 raw reward、centered reward 与 normalized reward：训练动态为什么会明显不同 5

5.1 课程为什么用排序任务做 toy experiment 5

5.2 直接用 raw reward，为什么效果一般 6

5.3 centered reward 为什么通常更稳 6

5.4 标准差归一化带来了什么，又引出了什么 7

5.5 本章小结 7

6 从 toy task 回到真实系统：为什么第 17 讲还在讲基础设施 7

6.1 RL 比 pretraining 更像“生成系统加反馈系统的耦合” 7

6.2 本章小结 7

7 课程最后想让你记住什么：RLVR 很强，但系统复杂度同样在上升 8

7.1 “If you can measure it, you can optimize it”的前提 8

7.2 为什么 RL systems 明显比 pretraining 更复杂 8

7.3 本章小结 8

8 总结与延伸 8

1 把语言模型写成一个 RL 问题：state、action、reward 到底是什么

1.1 第 17 讲先把符号和对象理顺

第 17 讲延续上一讲的 RLVR 主线，但这次不再讨论大局，而是专门拆机械细节。课程首先把语言模型训练写成一个近似 bandit / episodic RL 问题：

- 状态 s：prompt 加当前已生成的前缀；
- 动作 a：下一 token，或者简化记号下的整段回答；
- 奖励 $R(s,a)$：对整段回答的评分，在这一讲里重点是 outcome reward 和 verifiable reward；
- 转移 $T(s' \mid s,a)$：基本是确定性的，因为新状态就是旧前缀加上新 token。

课程特别提醒，这个设定和机器人 RL 很不一样。语言模型的状态并不是物理世界给你的，而是由文本轨迹构成；这意味着 planning 和 test-time compute 的自由度更大，但也意味着很多经典 RL 假设，例如连续折扣回报或复杂随机转移，在这里不再是中心。

第 17 讲的任务

不是再讲“为什么要做 RL”，而是把 RL 在语言模型里真正起作用的那一套数学和工程部件拆开，看清每一块到底解决什么问题。

1.2 本章小结

第 17 讲一上来先把语义讲清楚：语言模型里的 RL 不是把传统 RL 公式原封不动搬过来，而是把 prompt-response 生成过程重新解释成一个可优化的轨迹问题。

2 naive policy gradient：公式很短，方差很大

2.1 从目标函数到 log-derivative trick

课程从最基础的目标出发：

$$\mathbb{E}[R] = \int p(s)\,\pi(a \mid s)\,R(s,a).$$

对参数求梯度，经过标准的 log-derivative trick，可以得到

$$\nabla\mathbb{E}[R] = \mathbb{E}\left[\nabla \log \pi(a \mid s)\,R(s,a)\right].$$

这个式子非常漂亮，因为它告诉你：只要采样出一条回答，并知道它的奖励，就能用该回答的 log probability 梯度乘以奖励来更新模型。

课程故意把这一步讲得很慢，因为这是后面所有变体的根。无论是 PPO、GRPO 还是别的 policy gradient 算法，本质上都没有离开这个式子，只是在想办法把这个估计器做得更稳、更低方差、更少偏差。

2.2 为什么 naive 版本会很糟

课程给出一个很直观的语言模型设定：奖励是 0/1，只有答对才给 1，答错给 0。这样一来，naive policy gradient 几乎只会在少数正确样本上更新，而大量错误样本都不会告诉模型“你错在哪里”。这种极端稀疏的反馈会让训练信号非常噪。

在 RLHF 里，reward model 往往是连续的，所以这种稀疏性没那么明显；但在 RLVR 里，很多奖励恰恰就是硬判定。课程因此想让学生理解：**policy gradient 的核心难题，不是公式不会写，而是估计器太吵**。

2.3 本章小结

naive policy gradient 提供了最原始的更新规则，也暴露了最原始的问题：高方差、稀疏奖励和低样本效率。后面所有技巧，都在试图修这些洞。

3 baseline 与 advantage：为什么“比平均好多少”比“拿了多少分”更重要

3.1 baseline 的作用是降方差，而不是改目标

课程接下来引入 baseline。核心想法很简单：如果对同一个状态 s，整体奖励水平本来就很高，那么直接乘以原始奖励会混淆“这个回答本身有多好”和“这个 prompt 本来就容易拿高分”。

于是我们考虑

$$\nabla \log \pi(a \mid s)\big(R(s,a) - b(s)\big),$$

其中 $b(s)$ 只依赖状态，不依赖动作。课程特别强调，这种减法不会改变目标的期望，只会改变梯度估计的方差。

3.2 课程那个两状态例子到底想说明什么

课程给了一个非常好的例子：状态 s_1 里两个动作都分数很高，状态 s_2 里两个动作都分数较低。如果你只看原始分数，可能会以为 s_1 的次优动作仍然比 s_2 的最优动作更值得强化；但从“相对本状态平均水平高多少”来看，显然不是这样。

这正是 baseline 的意义。它不是在问“这次回答绝对得分多少”，而是在问“在这个 prompt 下，这条回答比通常水平好还是差”。对于语言模型来说，这种相对性非常关键，因为 prompt 难度差异巨大。

官方 `lecture_17.py` 甚至把这个例子做成了一个小算术：如果四个样本奖励分别是 `11, 9, 0, 2`，直接用原始奖励的标准差大约是 `5.323`；若对两个状态分别减去 baseline `10` 和 `1`，新的样本变成 `1, -1, -1, 1`，标准差会降到大约 `1.155`。课程借这个极小例子想让学生立刻看见：**baseline 的主要收益不是“更对”，而是“更稳”**。

3.3 advantage 其实就是一个更有语义的 baselined reward

课程进一步把 baseline 与 advantage 连接起来：

$$V(s) = \mathbb{E}[R \mid s], \qquad Q(s,a) = \mathbb{E}[R \mid s,a], \qquad A(s,a) = Q(s,a) - V(s).$$

如果选择 $b(s) = V(s)$，那么 $R - b(s)$ 就正好变成 advantage。它衡量的不是“这个动作拿了多少奖励”，而是“这个动作比当前状态下的平均动作好多少”。

为什么 advantage 是 RL 里的核心语言

因为它自动把“任务本身难不难”这部分剥掉了，留下的只是动作选择本身的相对价值。对语言模型来说，这就像是在不同难度 prompt 间做公平比较。

3.4 本章小结

baseline 和 advantage 让 policy gradient 从“被绝对奖励牵着走”变成“对相对优劣更敏感”。这是从可写公式走向可训练算法的第一步。

4 GRPO：把多回答组结构变成廉价 advantage

4.1 为什么语言模型天然适合 group-relative baseline

课程指出，语言模型和很多传统 RL 任务最大的不同之一是：对于同一个 prompt，我们可以非常自然地采样多条回答。于是，不用训练一个单独 value model，也能在组内估一个“平均水平”。这就是 GRPO 的出发点。

最简单的组内优势可以写成

$$\delta_i = \frac{R_i - \mu_g}{\sigma_g + \epsilon},$$

其中 R_i 是组内第 i 条回答的奖励，μ_g 和 σ_g 是该 prompt 对应组内奖励的均值和标准差。课程故意把它讲得很直接：这其实就是把 reward 做 z-score，然后把它当成 advantage 的近似。

Algorithm 1 Iterative Group Relative Policy Optimization

Input initial policy model $\pi_{\theta_{\text{init}}}$; reward models r_φ; task prompts $\mathcal{D}$; hyperparameters ε, β, μ

```
policy model π_θ ← π_θ_init
for iteration = 1, ..., I do
    reference model π_ref ← π_θ
    for step = 1, ..., M do
        Sample a batch D_b from D
        Update the old policy model π_θ_old ← π_θ
        Sample G outputs {o_i}_{i=1}^G ~ π_θ_old(· | q) for each question q ∈ D_b
        Compute rewards {r_i}_{i=1}^G for each sampled output o_i by running r_φ
        Compute Â_{i,t} for the t-th token of o_i through group relative advantage estimation.
        for GRPO iteration = 1, ..., μ do
            Update the policy model π_θ by maximizing the GRPO objective (Equation 21)
    Update r_φ through continuous training using a replay mechanism.
```

Output π_θ

图 1: GRPO 的算法骨架并不复杂：采样一组回答，算 reward，做组内相对归一化，再更新策略。

```
responses = model.sample(prompts, group_size=G)
rewards = verifier(responses)
advantages = (rewards - rewards.mean(dim=1, keepdim=True)) / (
    rewards.std(dim=1, keepdim=True) + 1e-4
)
loss = -(advantages.detach() * logprob_ratio_clipped).mean()
```

Listing 1: 极简 GRPO 风格伪代码

4.2 为什么旧策略的 log prob 必须被冻结

官方 `lecture_17.py` 在 walkthrough 里专门插了一个很容易被忽略的工程点：当 loss 里出现

$$\frac{\pi_\theta(a \mid s)}{\pi_{\text{old}}(a \mid s)}$$

这种 ratio 时，π_{old} 必须被当成常数，而不是继续让梯度流回去。否则，你以为自己在和“旧策略”比较，实际上却是在和一个也会一起移动的目标比较。

这也是为什么很多实现里都会显式 `detach`，或者在计算旧策略 log prob 时包上 `torch.no_grad()`。课程在这里故意给了一个极简例子：如果 `p_old` 不冻结，那么 ratio 对同一个参数的梯度会因为分子分母同时依赖它而发生抵消，更新语义完全变味。

GRPO/PPO 里非常容易犯的实现错误

看到 $\pi(a \mid s)/\pi_{\text{old}}(a \mid s)$ 并不等于真的在比较新旧策略。只有当旧策略被冻结时，这个 ratio 才有“信赖域比较器”的含义。

4.3 KL 正则为什么还要保留

课程同时提醒，GRPO 不是只算相对奖励就够了。你通常还要加显式 KL 正则，让新策略不要离参考策略跑太远。否则，模型可能为了追逐短期奖励迅速坍塌，忘掉原本的语言能力或进入奇怪的输出模式。

这也是课程反复强调“RL systems is much more complex than pretraining”的原因。即便公式看起来简单，真正的训练里仍然要管理 rollout、reference model、reward computation 和 KL control。

官方 script 这里还给了一个常用的 KL 近似形式：

$$\mathrm{KL}(p\|q) = \mathbb{E}_{x\sim p}\left[\frac{q(x)}{p(x)} - \log\frac{q(x)}{p(x)} - 1\right].$$

它的好处是，可以直接用当前策略样本和参考策略 log prob 做无偏或近似无偏的估计，而不用显式把整份分布枚举出来。对大语言模型来说，这种“只用采样轨迹估 KL”的技巧几乎是不可或缺的。

4.4 本章小结

GRPO 最吸引人的地方，在于利用语言模型天然的 group structure 省掉了 critic。但它仍然需要 reference policy、KL 正则和 careful reward design，绝不是“只有一行 loss”的神奇算法。

5 raw reward、centered reward 与 normalized reward：训练动态为什么会明显不同

5.1 课程为什么用排序任务做 toy experiment

第 17 讲用一个“给定若干数字，输出排序结果”的玩具任务来解释 GRPO，非常聪明。这个任务够简单，奖励函数也能明确写出，但训练动态已经能暴露出 policy gradient 的核心问题：如果奖励设计或基线选取不对，模型很容易停在局部最优。

官方 `lecture_17.py` 在这里其实给了两种 reward，而不是一种。第一种 `sort_distance_reward` 很硬，只奖励“第几个位置和排好序的答案完全一致”；第二种 `sort_inclusion_ordering_reward`

更软，一部分奖励来自元素有没有出现在响应里，另一部分来自相邻元素是否已经基本有序。这两个 reward 的差别非常关键，因为它说明：**verifiable reward 也不等于只有单一硬判定，你仍然可以设计更平滑的部分信用**。

为什么 toy task 要故意给两个 reward

因为 RLVR 真正难的往往不是“有没有 verifier”，而是 verifier 给出的信号到底是极端稀疏，还是能提供足够多的中间梯度方向。排序任务只是把这个问题缩小后让你看清楚。

5.2 直接用 raw reward，为什么效果一般

课程先展示了 raw reward 更新。它的特点是：高 reward 的回答得到正推动，但低 reward 的回答未必受到明确的负推动，于是训练容易显得“有信号，但不够锋利”。很多参数会沿着一个模糊方向移动，最后学出一个不差但也不够好的策略。

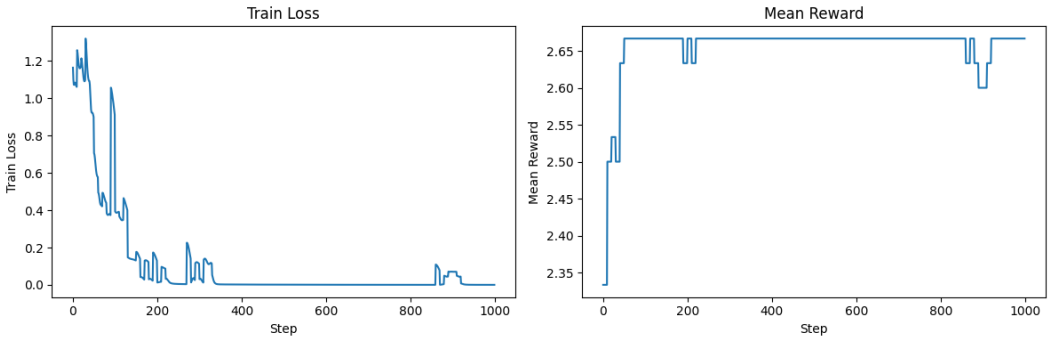

图 2: 直接用原始 reward 时，训练虽然会进步，但往往更容易卡在次优状态。

5.3 centered reward 为什么通常更稳

课程接着改用 centered reward，也就是先减去组内均值。这样一来，低于平均的回答会得到负更新，高于平均的回答会得到正更新；如果一组回答都一样差或一样好，则不更新。这个机制非常符合直觉，因为它把学习信号集中到“谁比同组其他回答更值得保留”上。

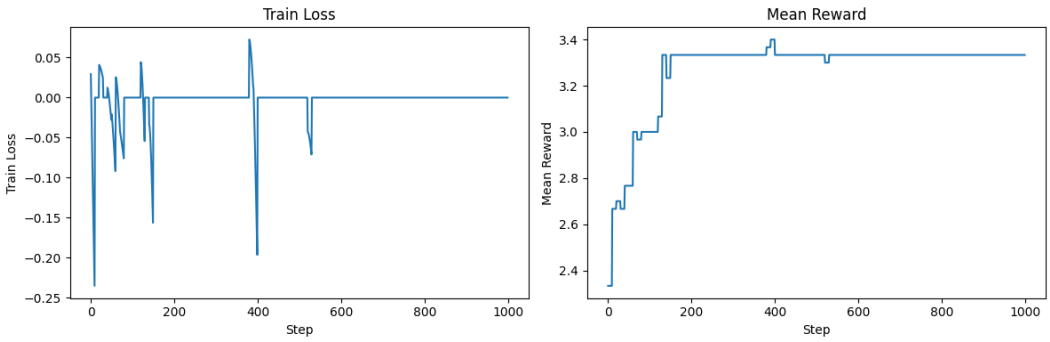

图 3: 组内中心化后，训练动态通常明显更稳，因为差回答会收到明确负信号。

5.4 标准差归一化带来了什么，又引出了什么

最后课程试了 normalized reward，也就是在中心化的基础上再除以组内标准差。直觉上，这会让“同样的绝对优势”在低方差组里被放大，在高方差组里被缩小，从而统一不同 prompt 的更新尺度。

但课程也提醒，这一步并不总是带来显著收益，甚至会引入长度偏置等额外问题。这也是后来一些变体，例如 Dr. GRPO，会对标准差归一化持更谨慎态度的原因。

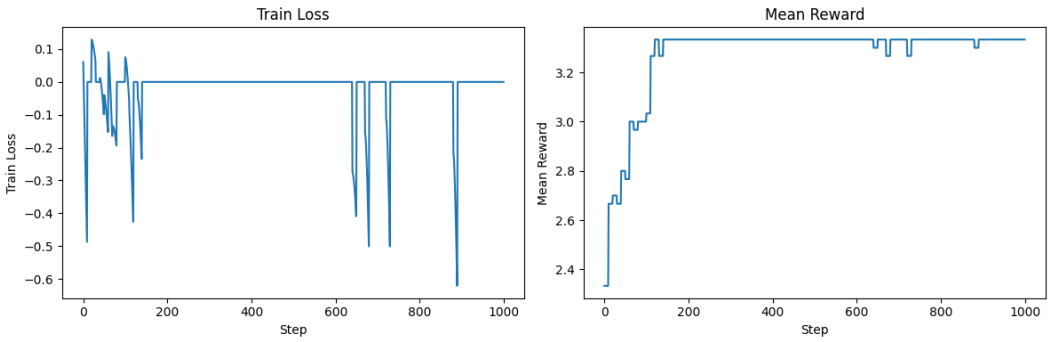

图 4: 标准差归一化有时更平滑，但不一定总比 centered reward 更好，也可能带来新的偏差。

5.5 本章小结

这组 toy experiment 最重要的教学价值，是让学生直观看到：reward 形状、baseline 和 normalization 不是实现细节，而是会实打实改写训练动态的核心设计。

6 从 toy task 回到真实系统：为什么第 17 讲还在讲基础设施

6.1 RL 比 pretraining 更像“生成系统加反馈系统的耦合”

虽然第 17 讲大部分时间都在讲公式，但官方结尾其实仍然回到了系统视角。原因很简单：你不是在静态数据上算 loss，而是在一个 rollout 系统里先生成，再打分，再更新。于是推理成本、参考模型、reward pipeline、缓存和批处理策略都会重新进入训练闭环。

这也是为什么课程最后反复说 RL systems 比 pretraining 更复杂。预训练阶段你只需要稳定地读数据、前向、反向；到了 RLVR 阶段，你同时在运行一个生成器、一个 verifier、一个参考策略和一个更新器。哪怕公式非常短，系统栈也会自然变厚。

6.2 本章小结

第 17 讲最后留下的不是“GRPO 已经解释完了”，而是一个更现实的认识：policy gradient 框架已经足够清楚，但要把它扩到真正的大模型推理训练，还需要很重的基础设施与 reward engineering。

7 课程最后想让你记住什么：RLVR 很强，但系统复杂度同样在上升

7.1 “If you can measure it, you can optimize it” 的前提

课程在最后把全局结论讲得很清楚：强化学习之所以成为下一代 reasoning 模型的重要引擎，是因为在一部分任务上，我们终于拿到了足够可测、可验证的奖励。但这句口号有隐藏前提，那就是你测的东西必须真的对，而且不能太容易被钻空子。

7.2 为什么 RL systems 明显比 pretraining 更复杂

第 17 讲最后一句很值得记住：RL systems 比 pretraining 复杂得多。原因不只是多一个 loss，而是整个训练对象变了。你需要做 rollout、保留 reference model、管理 reward pipeline、平衡 KL、处理长短不齐的轨迹，还要面对推理式工作负载带来的系统瓶颈。

对第 17 讲最容易产生的误读

不要因为 GRPO 的伪代码很短，就误以为 reasoning RL 已经“被彻底解决”了。课程的真实态度是：概念骨架已经很清楚，但好 reward、好基础设施和好训练细节仍然决定成败。

7.3 本章小结

第 17 讲最后把学生带回一个更成熟的位置：policy gradient 框架是清楚的，但把它稳定地用在大模型上，依然需要大量系统和算法工程。

8 总结与延伸

第 17 讲把 RLVR 的机械结构彻底拆开，给整门课做了一个很强的技术收束。

第一，**语言模型里的 RL 仍然从最基本的 policy gradient 出发**。只是状态、动作和奖励的语义与传统 RL 略有不同。

第二，**baseline 和 advantage 是降方差的核心**。它们让模型关注“相对本 prompt 平均水平的提升”，而不是被绝对奖励牵着走。

第三，**GRPO 利用了语言模型的组结构**。同一 prompt 采样多条回答，就天然提供了一个便宜的相对比较基线。

第四，**reward 形状会直接改写训练动态**。raw reward、centered reward 和 normalized reward 的差别不是数学装饰，而是实际影响收敛路径。

第五，**RLVR 的前提是 reward 足够可测**。一旦可以可靠验证，RL 才真正能成为能力增长引擎。

第六，**RL 系统复杂度远高于预训练**。rollout、reward、KL 和 reference model 让整个后训练栈变得更像一个在线优化系统。

整门 CS336 经过第 17 讲后的最终压缩

从零构建语言模型，最后不会停在 tokenization、Transformer 或 scaling law 上，而会自然走向一个更难的问题：当模型已经足够大时，我们还能给它什么样的反馈，才能持续把能力往前推。

到这里再回看课程开头，就会发现 CS336 的主线非常完整：先把语言模型做出来，再把它做大、做快、做稳、做可评测，最后再去思考如何让它在可验证目标上继续变强。